Groomee

Project Documentation · Generated 2026-05-21

Groomee is an on-demand beauty marketplace serving Lagos (live) and Abuja (waitlist). Customers book vetted beauty professionals to their home; professionals run their business from the partner portal; admins operate the marketplace, the pro economy (wallet, savings, benefits) and the consumables supply chain (products, inventory, recipes) from a permissioned dashboard. This document is a complete technical handover.

Source: `https://github.com/dodulana-tech/groomee` · Production: `https://groomee.onrender.com` · Owner email: `dodulana@gmail.com`

Table of contents

- Project overview
- Current build status
- Technical stack
- Architecture overview
- Features implemented
- Features not yet implemented
- Database schema
- API endpoints
- Third-party integrations
- Credentials and platforms
- Known issues and bugs
- Setup and installation
- Handover notes

1. Project overview

What it is

Groomee is an on-demand beauty professional marketplace. Customers book hair, makeup, nail, lash, barbing and skincare services to be delivered at their address, on-demand or scheduled, in cities the company has activated. The platform handles vetting, scheduling, dispatch, payments, refunds, reviews and ongoing care continuity, plus the financial wellbeing and supply chain of the pros themselves.

Target users

- **Customers** — Lagos (live) and soon Abuja (waitlist). Book single or chained services (e.g. loosen + wash + reloc), choose between ASAP or scheduled, optionally share a health profile for safer service delivery.
- **Beauty professionals (pros)** — Independent operators, masters with apprentices, or apprentices/staff under a master. Use a partner portal to manage availability, accept bookings, see calendar, run their growth tools. Earn through the wallet ledger, save through the Earnings Reserve, get HMO + accident cover through Benefits.
- **Internal operations / admins** — Role-based access to bookings, pros, customers, disputes, advances, payouts, subscriptions, catalog, surveys, waitlist, analytics, team management, settings, calendar, health PHI, city rollout, wallets, savings, benefits, products and inventory.

Revenue model

- 20% platform commission on every booking by default (configurable per pro via `commission` on the `Pro` row).
- Surcharges layered on top: emergency / ASAP (+20%), late-night (+25%), early-morning (+20%).
- Subscriptions: customer plans that bundle services at a discount, with `SubscriptionUsage` tracking redemptions.
- Gift cards: redeemable via codes; applied as a discount line item at booking creation.
- Pro advances: a financial product that pays out a portion of pending earnings early, governed by a credit score on the `Pro` record.
- Earnings Reserve (savings): pros opt in to skim a configurable percent of each payout into a vault, with notice-period withdrawals and a platform match bonus.
- Benefits programme: HMO and accident-cover plans with platform subsidy + pro premium contribution, debited from earnings on a recurring schedule.
- Consumables: products catalog drives a per-pro inventory and per-service recipe consumption.

Brand

- The customer-facing word is **pro** or **beauty professional**. The legacy word "groomer" is rejected and is being phased out (some legacy table mappings still carry the old name, e.g. `@@map("GroomerStatus")`).
- Design system: warm-neutral landing page, glassmorphism UI for the booking surfaces, dark admin shell, dark green partner portal.

2. Current build status

The product is in late-stage build. Lagos operations are live; Abuja is in waitlist mode and can be activated by a super admin without a deploy. Major recent slices include the multi-service scheduler, the health-and-care system, the city activation system and the pro-economy stack (wallet, savings, benefits, products, inventory).

Last 15 commits on `main`

```
ae23427  style: remove em-dashes from UI copy only
4380b27  Revert "style: replace every em-dash with a hyphen across the codebase
5e947b7  style: replace every em-dash with a hyphen across the codebase
7b569d7  fix(pwa): skip service-worker registration in dev (prevents stale-chun
e894ca4  feat(cities): city activation system + city-driven landing page
412f449  fix(deploy): bump SW cache + auto-sync admin role permissions on deplo
04f8864  test(health): extend deep-test with health profile coverage + wire ca
deb29cc  feat(health): customer health profiles + pro care briefs end-to-end
c7ad1d2  test: deep-test harness covering every Groomee feature
a26a590  feat(scheduler): multi-service line items (loosen + wash + reloc)
e6c054d  feat(scheduler): pro calendars + zone travel buffers + conflict rejec
568e63e  feat(home): merge categories + top-pros into Browse; move trust badges
ef1b980  feat(delegation): audit-log delegate and rescind events
1e13f20  fix(delegation): durable earnings split + protect master's other in-fl
86cee2a  feat(apprenticeships): slice 8 — booking delegation by master
```

Slice tracker

SLICE	STATUS	NOTES
Apprenticeships (8 slices)	Shipped	Invitation, training modules, sign-off, independence, freedom ceremony, certificates, earnings split, delegation.
Scheduler — calendars + travel + conflict rejection	Shipped	Per-pro day view (admin + partner), zone travel matrix, 409 on conflicts.
Multi-service line items	Shipped	`BookingItem` join model; primary `Booking.serviceId` kept for back-compat.
Health profile + care briefs	Shipped	PHI audit, customer editor, partner brief + acknowledge, admin profile viewer.
City activation system	Shipped	Schema-backed cities, `/admin/cities` toggle, landing page is now city-driven.
Wallet ledger + lifecycle releases	Shipped	`ProWallet` + `WalletLedger`; mobilization on accept, completion + confirm releases, clawback on refund, manual adjust + recompute, ledger CSV export.
Earnings Reserve (savings)	Shipped	`ProSavingsAccount` + `SavingsMovement` + `SavingsWithdrawal`; opt-in/out, configurable skim %, notice-period withdrawals with force-mature / release / retry / cancel, match-bonus, admin overview.
Benefits (HMO + accident)	Shipped	`BenefitPlan` / `BenefitSubsidyRule` / `ProBenefitEnrollment` / `BenefitDebit` / `BenefitClaim`; daily debit cron at 02:00 UTC; admin plans, claims, debits, eligibility, pause/resume.
Products + inventory + recipes	Shipped	`Product` + `ProInventory` + `InventoryTxn` + `ServiceRecipe`; admin catalog, per-pro inventory with adjust/restock/threshold, low-stock board, usage stats.
Hard delete for admin users	Deferred	Soft delete (deactivate to CUSTOMER) is in. Hard delete intentionally not built — preserves `ActivityLog` integrity.
Per-item earnings split (multi-service)	Deferred	Primary service drives commission today; sum of all items drives the chain total.
Scheduler week / month views	Deferred	Only the per-pro day view is shipped.
`ASK_PER_BOOKING` consent UI (health)	Stub	Endpoint treats it as no-share until the per-booking consent UI lands.

3. Technical stack

Runtime

LAYER	TECHNOLOGY	VERSION
Framework	Next.js (App Router, Turbopack)	16.1.6
Language	TypeScript	^5
UI	React	^18
Styling	Tailwind CSS (+ `@tailwindcss/postcss`)	3.4.19 / 4.2.0
Database client	Prisma + `@prisma/client`	^5.22.0
Database	PostgreSQL (Render managed)	—
Auth tokens	`jose` (JWT signing)	^5.3.0
Validation	`zod`	^3.23.8
Date helpers	`date-fns`	^3.6.0
Email	`nodemailer` over SMTP	^7.0.13
WhatsApp / SMS	`twilio`	^5.12.2
Class helpers	`clsx`, `tailwind-merge`	—
Script runner	`tsx`, `ts-node`	—

Dev tooling

- `eslint` + `eslint-config-next` + `eslint-config-prettier`
- `prettier` for formatting
- Turbopack as the dev compiler (via Next 16)

NPM scripts

SCRIPT	WHAT IT DOES
`npm run dev`	Start Next.js dev server.
`npm run build`	Production build.
`npm run start`	Run the production server.
`npm run db:generate`	Regenerate the Prisma client.
`npm run db:push`	Push schema to the connected database without migrations.
`npm run db:migrate`	`prisma migrate deploy` for managed migrations.
`npm run db:seed`	Seed roles, cities, zones, services, sample pros.
`npm run db:studio`	Open Prisma Studio.
`npm run sync:roles`	Idempotent additive-only sync of `DEFAULT_ROLES` permissions to the live DB. Used in the deploy build.
`npm run gc:temp-users`	Garbage-collect unverified temporary user rows.
`npm run smoke:admin`	HTTP smoke test for admin pages and APIs.
`npm run test:deep`	Long-running end-to-end harness covering every major feature.

4. Architecture overview

App Router route groups

Next.js App Router with three customer-facing route groups plus an admin shell:

GROUP	PURPOSE	LAYOUT
`(customer)`	Landing, search, booking, profile, gift, subscriptions, points, health profile.	Public Navbar + footer; PWA.
`(admin)`	Internal operations dashboard, gated by `role === "ADMIN"` in middleware.	Dark sidebar; permission-filtered nav with 22 entries.
`(partner)`	Beauty professional portal (bookings, calendar, earnings, profile, team).	Green sidebar + bottom nav on mobile.
`(partner-auth)`	Public onboarding flow for pros.	Minimal.

Layered architecture

- **API routes** (`src/app/api/**/route.ts`) — thin HTTP handlers that authenticate, validate (zod), call into `src/lib` helpers, and return JSON.
- **Library** (`src/lib/*`) — business logic. Core modules: `dispatch` , `scheduling` , `booking-items` , `cities` , `health` , `auth` , `paystack` , `whatsapp` , `email` , `surcharge` , `apprenticeships` , `auto-confirm` , `permissions` , `wallet` , `wallet-releases` , `savings` , `benefits` , `inventory` .
- **Database** (`prisma/schema.prisma`) — 57 models, 25 enums. Single Postgres instance.
- **Middleware** (`src/middleware.ts`) — gates protected paths, attaches user role headers, redirects on auth.
- **Edge auth helper** (`src/lib/auth-edge.ts`) — JWT verification compatible with the Edge runtime used by middleware.

Authentication

- Phone OTP or email OTP — OTP stored in `OtpCode` with a 10-minute expiry; old codes are auto-invalidated when a new one is issued.
- JWT signed with `JWT_SECRET` , set as an HTTP-only cookie.
- Session payload carries `userId` , `role` , `adminRoleId` , `adminRoleName` , `permissions[]` .
- Server-side revocation via `User.sessionsValidFrom` ; an in-memory cache (30 s TTL) coalesces revocation lookups.

- Roles: `CUSTOMER`, `ADMIN`, `PRO`. An ACTIVE Pro tied to the user upgrades a CUSTOMER session at sign time.

Booking lifecycle

1. **Create** — `POST /api/bookings`. Customer chooses primary service + optional additional services + zone + ASAP/scheduled + optional pro. Server validates lead time, city is LIVE, working-hours and conflict-window for the chosen pro (if scheduled), then creates the booking + items + Payment row + (optionally) deducts points + redeems gift card — all in one transaction.
2. **Pay** — Customer pays on Paystack; webhook verifies; Payment status flips to `AUTHORISED`.
3. **Dispatch** — `tryNextPro` offers the job to ranked candidates; on accept the pro flips to `BUSY` + mobilization is released into their wallet + a care-notes email fires if applicable.
4. **En route** → **arrived** → **in-service** → **completed** — pro advances state. `COMPLETION_RELEASE` posts to the wallet at COMPLETED.
5. **Confirm** — customer confirms. `CONFIRM_RELEASE` posts the final earnings slice to the wallet; apprentice-commission release fires for the master if applicable; payment captured; pro flips back to `ONLINE`.
6. **Refund / cancel** — reverses any wallet movements with a `CLAWBACK` entry.
7. **Review / dispute** — review window opens; disputes routed to `/admin/disputes`.

Pro economy (wallet, savings, benefits)

- **Wallet (`ProWallet` + `WalletLedger`)** — append-only ledger of every monetary movement against a pro's balance. Movement types: `MOBILIZATION`, `COMPLETION_RELEASE`, `CONFIRM_RELEASE`, `APPRENTICE_COMMISSION_RELEASE`, `CLAWBACK`, `ADVANCE_DISBURSED`, `ADVANCE_REPAYMENT`, `PAYOUT_DEBIT`, `PAYOUT_REVERSAL`, `MANUAL_ADJUSTMENT`, `SAVINGS_SKIM`, `BENEFIT_PREMIUM`. Admins can manually adjust + recompute the running balance and export the ledger as CSV.
- **Earnings Reserve (`ProSavingsAccount` + `SavingsMovement` + `SavingsWithdrawal`)** — pros opt in (default on) to skim a configurable percent of each payout into a vault. Withdrawals respect a notice period; admins can force-mature, release, retry or cancel. Match-bonus credits and admin adjustments are first-class movements.
- **Benefits (`BenefitPlan` / `BenefitSubsidyRule` / `ProBenefitEnrollment` / `BenefitDebit` / `BenefitClaim`)** — HMO and accident cover. Plans carry tiers + premiums + subsidies; enrolments are debited monthly by a 02:00 UTC cron that is idempotent per `(enrolment, period)`; pause/resume; grace period and PAUSED → CANCELLED transitions; claims tracked end-to-end.

Consumables (products + inventory + recipes)

- **Products** — global catalog with categories, unit cost, usage stats.
- **ProInventory** — per-pro stock levels with low-stock thresholds.
- **InventoryTxn** — audit trail of every adjust/restock/consume; reason codes drive the txn list filter.
- **ServiceRecipe** — declarative consumption: "doing service X consumes N units of product Y". When a booking is completed, inventory is automatically decremented per the recipe.

Apprenticeship model

- Three relationship modes: `INDEPENDENT`, `APPRENTICE`, `STAFF`.
- A master invites an apprentice; modules are signed off; a "Freedom ceremony" graduates them; a public certificate is issued.
- The master earns a configured commission on dependent activity until Freedom; delegation lets the master deploy a dependent on a booking. Commission posts as `APPRENTICE_COMMISSION_RELEASE` in the wallet ledger.

City activation

- The `City` model carries operational state (`HIDDEN` / `WAITLIST` / `LIVE`).
- Zones link to a city. `POST /api/bookings` rejects bookings into non-LIVE zones with `409 CITY_NOT_LIVE`.
- The landing page is fully driven by City data: hero CTA, city rollout banner, footer, JSON-LD `areaServed` all derive from the table.
- A super admin can flip a city LIVE from `/admin/cities`; this fires a best-effort WhatsApp/email blast to the existing waitlist.

Hosting topology

- **Web service** — the Next.js standalone build runs on Render.
- **Database** — `groomee-db` Postgres on Render.
- **Cron 1** — every 10 minutes: `GET /api/cron/tick` (auto-confirm, dispatch retries).
- **Cron 2** — daily at 06:00: `GET /api/cron/renew-subscriptions`.
- **Cron 3** — daily at 02:00 UTC: `GET /api/cron/benefits` (idempotent per enrolment/period; powers debits, grace nudges, PAUSED → CANCELLED).
- **Build command** includes `prisma db push --accept-data-loss` and `npm run sync:roles` so schema and role permissions stay in sync on each deploy.

5. Features implemented

Customer surfaces

- Landing page with city-driven CTAs, hero booking quick-search, browse-by-category, top pros grid.
- Search: filter by service, zone, ASAP, availability.
- Pro profile page with services, zones, reviews, BookingPanel for direct booking.
- BookingPanel: multi-service chaining (primary + add-ons), real availability slots fetched from the scheduler (respecting working hours, travel buffers, conflicts), surcharge preview, points + gift-card redemption, live health-contraindication preview.
- Customer booking detail page with live status, line items, total duration, location, tracker.
- Customer bookings list page.
- Customer auth: phone OTP or email OTP, link a phone to an email-only account.
- Profile dashboard with sub-pages: edit profile, beauty profile, health & sensitivities, my squad (favourites), my points + ledger, subscriptions, gift cards.
- Survey + city waitlist (generic, multi-city) capture.
- Gift a Glow-up: purchase, redeem.
- PWA: installable, manifest, service worker (production only — dev disables to avoid stale chunk issues).

Pro / partner portal

- Pro onboarding: phone OTP, profile, services, zones, photo, bank, schedule.
- Dashboard with stats.
- Bookings list with status filters, status advancement, delegation, rescind.
- Care brief expander per booking: severity-pinned rules, customer notes, condition list, one-tap acknowledge that freezes a snapshot.
- Calendar: per-day view with status-coloured blocks and dashed travel buffers.
- Earnings + advances request.
- Schedule (weekly working hours) editor.
- Team: invite, train, sign off, grant independence, grant Freedom; certificate page; delegation.
- Growth (referral / personal performance) page.
- Profile + bank verify.

Admin portal — 22 sidebar entries

AREA	PERMISSION	WHAT IT DOES
Overview	dashboard.view	Top-line stats.
Bookings	bookings.view / bookings.manage / bookings.refund	List, detail, assign, refund, override.
Calendar	pros.view	Per-pro day view with travel ghosts.
Beauty Pros	pros.view / pros.manage	List, detail, calendar, edit, verify, strike.
Apprenticeships	apprenticeships.view / apprenticeships.manage	Oversight, force-freedom, terminate.
Health & care	health.view / health.manage	Contraindication rule editor, PHI access log, profile list and detail (audit-logged).
Cities	cities.view / cities.manage	Activate / Waitlist / Hide, tagline editor, add new city.
Customers	customers.view / customers.manage	List, detail.
Disputes	disputes.view / disputes.manage	Queue + resolution.
Advances	advances.view / advances.manage	Approve / deny.
Payouts	payouts.view / payouts.manage	List, retry, history.
Wallets	wallet.view / wallet.manage	Overview, per-pro ledger, CSV export, manual adjust, recompute balances, fraction settings.
Savings	savings.view / savings.manage	Overview, accounts, per-pro opt-in/out + rate + goals + match-bonus + verify balances, withdrawals with force-mature/release/retry/cancel, config, movements CSV.
Benefits	benefits.view / benefits.manage	Plans (with subsidy rules), enrollments (attach-policy/pause/resume/cancel/debit-now), claims, debits, eligibility per-pro, config, overview.
Subscriptions	subscriptions.view / subscriptions.manage	Customer plan management.
Services	catalog.view / catalog.manage	Service catalog + per-service recipe (consumables).
Products	products.view / products.manage	Global product catalog, unit cost, usage stats.

Inventory	inventory.view / inventory.manage	Per-pro stock, adjust, restock, threshold, low-stock board, txn log.
Surveys	analytics.view	Survey responses.
Waitlist	analytics.view	City waitlist viewer.
Analytics	analytics.view / analytics.export	Stats + CSV export.
Team	team.view / team.manage	Invite admins, role editor, soft-delete.
Settings	settings.view / settings.manage_ops / settings.manage_finance	Key/value runtime config, including the travel-time matrix.

Cross-cutting

- Audit log (`ActivityLog`) for admin actions.
- Permission registry (`src/lib/permissions.ts`) with default roles seeded into the DB and refreshed on deploy by `sync:roles` .
- Notifications: WhatsApp / SMS (Twilio) and email (Nodemailer / SMTP). Care notes email fires from both dispatch accept and admin assign when relevant.
- Scheduler library: `findConflict` with travel-buffer padding, `getAvailableSlots` , `inWorkingHours` , `travelMinsBetween` .
- Booking-items library: `resolveBookingServices` sums durations + prices for chained services.
- Booking enforcement: city LIVE gate, conflict 409, lead-time validation, working-hours validation.
- Wallet releases library: `releaseMobilization` , `releaseCompletion` , `releaseConfirm` , `clawback` — all transactional + idempotent per booking-and-stage.
- Benefits debit runner: idempotent per `(enrolment, period)` so the daily cron is safe to fire repeatedly.

6. Features not yet implemented

- **Hard delete for admin users.** Soft delete only (PATCH with `deactivate: true` demotes the row to `CUSTOMER`). Intentional — keeps audit trail intact. Adding a true DELETE means orphaning `ActivityLog.adminId` references.
- **Per-item earnings split.** Multi-service bookings sum into a single `proEarning` on the parent Booking row. Splitting commission per `BookingItem` is deferred until ops requests it.
- **Scheduler week and month views.** Only the per-pro day view is shipped. Drag-to-reschedule is also deferred.
- **All-pros aggregate calendar.** Dispatcher would benefit from a single board showing every pro's day at once. Not built.
- **Per-booking consent for `ASK_PER_BOOKING` health visibility.** The endpoint currently treats `ASK_PER_BOOKING` as no-share (with a TODO note). The full per-booking consent UI is deferred.
- **Soft launch city state.** Three states (`HIDDEN` / `WAITLIST` / `LIVE`) are intentional. A fourth (limited / invite-only) can be added later if a real ops use case appears.
- **Per-city pricing differences.** Already partially handled by zone-level service pricing; explicit per-city rules are not modelled.
- **Background job queue.** Notifications and best-effort tasks (waitlist blast, care-notes email) run fire-and-forget inside the request lifecycle. There is no durable retry / queue yet.
- **Apprenticeship "Pause" state.** Today, apprenticeships flow IN_TRAINING → READY_FOR_FREEDOM → FREED → TERMINATED. A pause state (medical leave, etc.) is not modelled.
- **Customer hard-delete and right-to-be-forgotten.** Deactivation is via the team route, not yet a customer-driven GDPR / NDPR flow.
- **HMO partner integration.** Benefits enrollments carry `attach-policy` mechanics but the actual integration with an HMO provider for live policy issuance / claims processing is operationally manual today.
- **Inventory consumption from completion.** Recipes exist; automated decrement on booking complete is wired in the lib but should be smoke-tested on real bookings before relying on it.

7. Database schema

Enums (25)

ENUM	VALUES
`UserRole`	CUSTOMER, ADMIN, PRO
`Gender`	MALE, FEMALE, NON_BINARY
`BookingStatus`	PENDING_PAYMENT, DISPATCHING, NO_GROOMER, ACCEPTED, EN_ROUTE, ARRIVED, IN_SERVICE, COMPLETED, CONFIRMED, CANCELLED, DISPUTED
`PaymentStatus`	PENDING, AUTHORISED, CAPTURED, REFUNDED, FAILED
`DisputeStatus`	OPEN, UNDER_REVIEW, RESOLVED, CLOSED
`ProStatus`	PENDING, ACTIVE, SUSPENDED, REMOVED
`ProAvail`	ONLINE, BUSY, OFFLINE
`ProRelationship`	INDEPENDENT, APPRENTICE, STAFF
`ApprenticeshipStatus`	PENDING_ACCEPT, IN_TRAINING, READY_FOR_FREEDOM, FREED, TERMINATED
`EarningType`	SERVICE, APPRENTICE_COMMISSION, TIP, ADJUSTMENT
`ServiceCategory`	HAIR, MAKEUP, NAILS, BARBING, LASHES, SKINCARE, OTHER
`HealthCategory`	SKIN_SENSITIVITY, ALLERGY, MOBILITY, REPRODUCTIVE, CHRONIC, MEDICATION, OTHER
`HealthSeverity`	MILD, MODERATE, SEVERE
`HealthVisibility`	ALWAYS_SHARE, ASK_PER_BOOKING, PRIVATE
`ContraindicationLevel`	INFO, WARN, BLOCK
`CityStatus`	HIDDEN, WAITLIST, LIVE
`WalletMovementType`	MOBILIZATION, COMPLETION_RELEASE, CONFIRM_RELEASE, APPRENTICE_COMMISSION_RELEASE, CLAWBACK, ADVANCE_DISBURSED, ADVANCE_REPAYMENT, PAYOUT_DEBIT, PAYOUT_REVERSAL, MANUAL_ADJUSTMENT, SAVINGS_SKIM, BENEFIT_PREMIUM
`SavingsMovementType`	SKIM, WITHDRAWAL_REQUEST, WITHDRAWAL_RELEASED, WITHDRAWAL_CANCELLED, MATCH_BONUS, INTEREST_CREDIT, ADJUSTMENT
`SavingsWithdrawalStatus`	PENDING, ELIGIBLE, PAID, CANCELLED, FAILED

`BenefitKind`	HMO, ACCIDENT (and possibly more in schema; verify before relying)
`BenefitEnrollmentStatus`	ACTIVE, PAUSED, CANCELLED, GRACE
`BenefitDebitStatus`	PENDING, PAID, FAILED, SKIPPED
`BenefitClaimStatus`	OPEN, IN_REVIEW, APPROVED, REJECTED, PAID
`ProductCategory`	HAIR_CARE, SKIN_CARE, NAIL_CARE, TOOLS, OTHER (verify in schema)
`InventoryTxnReason`	RESTOCK, ADJUST, CONSUME_BOOKING, WRITE_OFF (verify in schema)

Models (57)

MODEL	PURPOSE
`User`	Single auth identity; can hold ADMIN, CUSTOMER or PRO role. Points balance, profile, sessionsValidFrom.
`AdminRole`	Named permission bundle; `permissions: string[]`; `isSystem` protects defaults.
`ActivityLog`	Audit trail for admin actions.
`Note`	Internal notes attached to entities (booking, customer, pro).
`Pro`	Beauty professional; links to `User`; status, availability, relationship, parent (master), commission rate, ratings, credit score.
`ProService`	Many-to-many: pro ↔ service with optional custom price.
`ProZone`	Many-to-many: pro ↔ zone coverage.
`ProSchedule`	Pro's weekly working window per day.
`Apprenticeship`	Master-apprentice relationship + lifecycle.
`CurriculumModule`	Module under an apprenticeship; gates independence.
`FavouritePro`	Customer's "squad" of favourite pros.
`Service`	Service catalog row: name, slug, category, price range, duration, sort order.
`ServiceRecipe`	Declarative consumables per service (consumes N of product Y).
`Zone`	Service area; linked to a City; both legacy `city: String` and `cityId: String?` kept for back-compat.
`City`	Operational region with status, launchedAt, tagline, sortOrder.
`ZoneTravel`	Zone-to-zone travel-minute matrix (admin-editable).
`Booking`	Primary booking row: customer, pro (nullable), primary service, zone, address, scheduledFor, durationMins (cached), pricing, status, lifecycle timestamps, delegated apprenticeship id.
`BookingItem`	Additional services chained to a booking (durationMins cached at create time).
`DispatchLog`	Per-pro offer log within a booking dispatch attempt.
`Payment`	One-to-one with Booking; tracks Paystack lifecycle.
`Dispute`	Customer-raised dispute on a booking.
`Review`	Customer review after a completed booking.

`Earning`	Pro earnings ledger (legacy; the wallet ledger is becoming canonical).
`Payout`	Bank payout to a pro.
`ProAdvance`	Advance request and lifecycle.
`ProCreditEvent`	Inputs to a pro's credit score.
`Setting`	Key/value runtime config (lead-time, surcharge windows, fraction tuning, etc.).
`Notification`	In-app notification.
`OtpCode`	Single-use OTP, 10-min expiry, auto-invalidated on a fresh issuance.
`SurveyResponse`	Customer survey submissions.
`Waitlist`	City waitlist signups.
`BeautyProfile`	Optional customer beauty profile (hair type, allergies, etc.).
`HealthProfile`	PHI: visibility, notes, conditions.
`HealthCondition`	Controlled-vocab condition row per profile.
`ServiceContraindication`	Rule: condition + (optional) service → INFO/WARN/BLOCK message.
`HealthAcknowledgment`	One per booking; JSON snapshot of profile + conditions frozen at ack time.
`HealthAccessLog`	PHI read audit; written by every brief and admin view.
`SubscriptionPlan`	Plans available for customer subscriptions.
`Subscription`	Customer subscription instance.
`SubscriptionUsage`	Each redemption of a subscription benefit on a booking.
`GiftCard`	Single-use code; redeem state.
`Referral`	Customer referral relationships.
`Strike`	Pro disciplinary record.
`PointsLedger`	Customer points entries.
`ProWallet`	Per-pro running balance. Source of truth for the wallet.
`WalletLedger`	Append-only wallet movements with a `WalletMovementType`.
`ProSavingsAccount`	Pro's Earnings Reserve: balance, skim rate, opt-in flag, goal, match-bonus config.
`SavingsMovement`	Append-only savings movements.
`SavingsWithdrawal`	Withdrawal lifecycle (notice period, eligibility, payout, retry, cancel).

`BenefitPlan`	Catalog of HMO/accident plans with premium tiers.
`BenefitSubsidyRule`	Platform subsidy rules per plan (eligibility → percent off premium).
`ProBenefitEnrollment`	Pro's active / paused / cancelled enrolment; carries policy attachment + grace period state.
`BenefitDebit`	One per (enrolment, period); idempotent.
`BenefitClaim`	Claim lifecycle: open → in-review → approved/rejected → paid.
`Product`	Global consumables catalog: name, category, unit cost.
`ProInventory`	Per-pro stock level and low-stock threshold.
`InventoryTxn`	Append-only inventory movements with a reason code.

8. API endpoints

All routes live under `src/app/api/**/route.ts`. Methods are derived from the exported functions. Authentication is read from the JWT cookie via `getSession()`. Admin routes are additionally gated by `hasPermission()` with permission strings registered in `src/lib/permissions.ts`.

Public

METHOD	ROUTE	PURPOSE
POST	`/api/auth/send-otp`	Issue a phone OTP. Customer or admin purpose.
POST	`/api/auth/verify-otp`	Verify phone OTP, set session cookie.
POST	`/api/auth/send-email-otp`	Issue an email OTP.
POST	`/api/auth/verify-email-otp`	Verify email OTP.
POST	`/api/auth/link-phone`	Attach a phone number to a verified account.
POST	`/api/auth/update-profile`	Edit user name / basic profile fields.
POST	`/api/auth/logout`	Clear session cookie.
GET	`/api/auth/me`	Return the current session payload.
GET	`/api/cities`	Public list of LIVE + WAITLIST cities.
GET	`/api/zones`	Public list of zones.
GET	`/api/services`	Public service catalog.
GET	`/api/pros`	Public list of pros (gated by apprenticeship independence).
GET	`/api/pros/[id]`	Single pro detail.
GET	`/api/pros/[id]/availability`	Slot picker (accepts repeated `?serviceId=`).
POST	`/api/surveys`	Submit survey response.
POST	`/api/waitlist`	Join a city waitlist.
GET	`/api/health`	Render health-check endpoint.
POST	`/api/webhooks/paystack`	Paystack webhook (signed).
POST	`/api/webhooks/whatsapp`	Twilio WhatsApp webhook.

Customer

METHOD	ROUTE	PURPOSE
GET / POST	`/api/bookings`	List own bookings / create a booking (city-LIVE gate, scheduler conflict gate, lead-time gate).
GET	`/api/bookings/[id]`	Detail (owner or admin).
POST	`/api/bookings/[id]/cancel`	Customer cancel.
POST	`/api/bookings/[id]/confirm`	Customer confirms; capture payment + record earning + wallet confirm-release.
POST	`/api/bookings/[id]/dispute`	Open a dispute.
POST	`/api/bookings/[id]/review`	Leave a review.
GET	`/api/bookings/[id]/payment-url`	Re-issue Paystack URL if the original expired.
POST	`/api/bookings/[id]/delegate`	Master delegates to a dependent.
POST	`/api/bookings/[id]/rescind`	Master rescinds delegation.
POST	`/api/bookings/[id]/assign`	Customer-side assign path.
GET / PUT	`/api/profile/health`	Read / update health profile (visibility, notes).
POST	`/api/profile/health/conditions`	Add a condition.
PATCH / DELETE	`/api/profile/health/conditions/[id]`	Edit / delete a condition.
GET	`/api/profile/health/catalog`	Controlled-vocab condition catalog.
POST	`/api/profile/health/check`	Preview contraindications for a list of services.
GET	`/api/profile/points`	Points balance and ledger.
GET	`/api/profile/referral`	Referral code and stats.
GET	`/api/profile/squad`	Customer favourites.
GET / POST	`/api/profile/beauty`	Beauty profile read / write.
GET	`/api/subscriptions`	List plans + own subscription.

POST	`/api/subscriptions/[id]/activate`	Activate a plan.
POST	`/api/subscriptions/[id]/cancel`	Cancel.
POST	`/api/gifts` + `/api/gifts/[id]/confirm` + `/api/gifts/redeem`	Gift-card purchase, confirm, redeem.
POST	`/api/payments/preview`	Quote a booking total without committing.
GET	`/api/payments/verify`	Paystack callback verifier.

Partner (pro)

METHOD	ROUTE	PURPOSE
POST	`/api/partner/onboarding`	Create pro profile / progress onboarding.
GET / POST	`/api/partner/profile`	Read / update pro profile.
POST	`/api/partner/photo`	Upload pro photo.
POST	`/api/partner/verify-bank`	Verify bank account.
GET / PUT	`/api/partner/schedule`	Weekly working hours.
GET	`/api/partner/dashboard`	Stats payload.
GET	`/api/partner/earnings`	Earnings ledger.
GET	`/api/partner/calendar`	Pro's own day calendar.
GET	`/api/partner/bookings`	Held + dependent bookings.
GET	`/api/partner/bookings/[id]/care-brief`	Live or frozen PHI brief.
POST	`/api/partner/bookings/[id]/care-brief/acknowledge`	Freeze acknowledgment snapshot.
POST	`/api/partner/apprentices/invite`	Invite a dependent.
POST / GET / PATCH	`/api/partner/apprentices/[id]/*`	Accept, decline, cancel, independence, freedom, modules, module patch.
POST	`/api/pro/advance`	Request an earnings advance.
GET	`/api/pro/credit-score`	Pro's credit score breakdown.

Admin — bookings, ops, catalog, team

METHOD	ROUTE	PERMISSION
GET	`/api/admin/stats`	dashboard.view
GET	`/api/admin/activity`	activity.view
GET / PATCH	`/api/admin/bookings` + `[id]` + `/assign` + `/refund`	bookings.* / bookings.refund
GET / POST / PATCH	`/api/admin/pros/list`, `/api/admin/pros/[id]`, `/api/admin/pros/[id]/calendar`	pros.*
GET / PATCH	`/api/admin/apprenticeships` + `[id]` + `/force-freedom` + `/terminate`	apprenticeships.*
GET	`/api/admin/disputes` + `[id]`	disputes.*
GET / PATCH	`/api/admin/advances` + `[id]`	advances.*
GET / POST	`/api/admin/payouts` + `/[id]/retry` + `/history`	payouts.*
GET	`/api/admin/subscriptions` + `[id]`	subscriptions.*
PATCH	`/api/admin/catalog/[id]`	catalog.manage
GET	`/api/admin/zones`	catalog.view
GET / POST / PATCH	`/api/admin/cities` + `/api/admin/cities/[slug]`	cities.*
GET / PUT	`/api/admin/travel-times`	settings.manage_ops
GET / POST / PATCH	`/api/admin/roles` + `[id]`	team.manage
GET / POST / PATCH	`/api/admin/team` + `[id]`	team.*
GET / PUT	`/api/admin/settings`	settings.*
GET	`/api/admin/analytics/export`	analytics.export
GET	`/api/admin/notes`	notes.*
POST / PATCH	`/api/admin/services/[id]/recipe` + `/[recipeId]`	catalog.manage

Admin — health (PHI)

METHOD	ROUTE	PERMISSION
GET / POST / PATCH	`/api/admin/health/contraindications` + `[id]`	health.* (every read audit-logged)
GET	`/api/admin/health/access-logs`	health.view
GET	`/api/admin/health/profiles` + `[userId]`	health.view (every read audit-logged)

Admin — wallets

METHOD	ROUTE	PERMISSION
GET	`/api/admin/wallet/overview`	wallet.view
GET	`/api/admin/wallet/movements`	wallet.view
GET / PUT	`/api/admin/wallet/settings`	wallet.manage
GET	`/api/admin/wallet/[proId]`	wallet.view
POST	`/api/admin/wallet/[proId]/adjust`	wallet.manage
POST	`/api/admin/wallet/[proId]/recompute`	wallet.manage
GET	`/api/admin/wallet/[proId]/ledger.csv`	wallet.view

Admin — savings (Earnings Reserve)

METHOD	ROUTE	PERMISSION
GET	`/api/admin/savings/overview`	savings.view
GET	`/api/admin/savings/accounts` + `/[proId]`	savings.view
POST	`/api/admin/savings/accounts/[proId]/adjust`	savings.manage
PUT	`/api/admin/savings/accounts/[proId]/goals`	savings.manage
POST	`/api/admin/savings/accounts/[proId]/match-bonus`	savings.manage
POST	`/api/admin/savings/accounts/[proId]/opt-in` / `opt-out`	savings.manage
PUT	`/api/admin/savings/accounts/[proId]/rate`	savings.manage
POST	`/api/admin/savings/accounts/[proId]/verify-balances`	savings.manage
GET / PUT	`/api/admin/savings/config`	savings.*
GET	`/api/admin/savings/movements.csv`	savings.view
GET	`/api/admin/savings/withdrawals`	savings.view
POST	`/api/admin/savings/withdrawals/[id]/cancel` / `force-mature` / `release` / `retry`	savings.manage

Admin — benefits (HMO + accident)

METHOD	ROUTE	PERMISSION
GET	`/api/admin/benefits/overview`	benefits.view
GET / POST / PATCH	`/api/admin/benefits/plans` + `[id]` + `/[id]/subsidies`	benefits.*
GET / POST / PATCH	`/api/admin/benefits/enrollments` + `[id]`	benefits.*
POST	`/api/admin/benefits/enrollments/[id]/attach-policy`	benefits.manage
POST	`/api/admin/benefits/enrollments/[id]/pause` / `resume` / `cancel`	benefits.manage
POST	`/api/admin/benefits/enrollments/[id]/debit-now`	benefits.manage
GET	`/api/admin/benefits/debits`	benefits.view
GET / PATCH	`/api/admin/benefits/claims` + `[id]`	benefits.*
GET	`/api/admin/benefits/eligibility/[proId]`	benefits.view
GET / PUT	`/api/admin/benefits/config`	benefits.*

Admin — products + inventory

METHOD	ROUTE	PERMISSION
GET / POST / PATCH	`/api/admin/products` + `[id]`	products.*
GET	`/api/admin/products/[id]/usage-stats`	products.view
GET	`/api/admin/inventory/overview`	inventory.view
GET	`/api/admin/inventory/low-stock`	inventory.view
GET	`/api/admin/inventory/txns`	inventory.view
GET / PUT	`/api/admin/inventory/settings`	inventory.*
GET	`/api/admin/inventory/[proId]`	inventory.view
POST	`/api/admin/inventory/[proId]/adjust`	inventory.manage
POST	`/api/admin/inventory/[proId]/restock`	inventory.manage
PUT	`/api/admin/inventory/[proId]/threshold`	inventory.manage

Cron

METHOD	ROUTE	SCHEDULE
GET	`/api/cron/tick`	Every 10 minutes (auto-confirm, dispatch retries).
GET	`/api/cron/renew-subscriptions`	Daily at 06:00.
GET	`/api/cron/benefits`	Daily at 02:00 UTC. Idempotent per `(enrolment, period)` so daily firing produces at most one debit per month; the daily cadence powers grace nudges and PAUSED → CANCELLED transitions.

All crons require a Bearer token equal to `CRON_SECRET` (generated by Render at first deploy).

9. Third-party integrations

SERVICE	WHAT IT DOES	ENV KEYS
Paystack	Card and bank payments. Authorize-and-hold model: payment captured only after the customer confirms service rendered. Bank transfers used for pro payouts and savings withdrawals.	`PAYSTACK_SECRET_KEY`, `NEXT_PUBLIC_PAYSTACK_PUBLIC_KEY`, `PAYSTACK_WEBHOOK_SECRET`
Twilio	WhatsApp and SMS for OTPs, pro job offers, customer status updates, and the city-launch waitlist blast.	`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_PHONE_NUMBER`, `TWILIO_WHATSAPP_NUMBER`
SMTP provider	Transactional email via Nodemailer (booking confirmations, care notes, apprentice events, gift cards, benefit debits/claims).	`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`
Cloudinary	Image hosting (referenced as a remote pattern in `next.config.mjs`).	Configured client-side via direct upload presets, no server env.
Render	Web + Postgres + three crons; see `render.yaml`.	`DATABASE_URL` injected from `groomee-db`; `CRON_SECRET` generated.
GitHub	Source control; auto-deploy trigger to Render on push to `main`.	—
HMO partner (TBD)	Benefits enrolments carry an `attach-policy` action that today represents a manual step on the partner's side. A real integration is not wired.	—

Legacy: the original README mentions Termii for WhatsApp + Supabase/Neon for DB + Vercel hosting. None of those are current. Twilio + Render Postgres + Render hosting are the live integrations.

10. Credentials and platforms

No passwords or tokens are listed below. Account owner is the email shown.

PLATFORM	ACCOUNT EMAIL	WHAT IT OWNS
GitHub	`dodulana@gmail.com` (user: `dodulana-tech`)	Repository `dodulana-tech/groomee`; auto-deploys main to Render.
Render	`dodulana@gmail.com`	Web service `groomee`, database `groomee-db`, crons `groomee-tick`, `groomee-renew-subs`, `groomee-benefits`. Production URL: `https://groomee.onrender.com`.
Custom domain	—	`groomee.ng` referenced in JSON-LD; check Render dashboard for DNS / SSL state before treating as live.
Paystack	Business owner email (likely `hello@groomeeapp.com`)	Live and test secret keys stored as Render env vars.
Twilio	Business owner email	SID + auth token + phone number + WhatsApp sender.
SMTP	Likely `hello@groomeeapp.com`	Transactional email outbound.
Business email	`hello@groomeeapp.com`	Public contact + Twitter / Instagram / TikTok / WhatsApp links (in Footer.tsx).
Local-dev admin	`dodulana@gmail.com` + phone `+2349138138553`	Super Admin account in the local DB only; do not assume it exists in production.
Seed super admin	Configured via `ADMIN_PHONES` + `ADMIN_EMAIL`	Default: `kiitan@groomeeapp.com` / `+2347067921202`. Override via env.

Environment variables in use

The local `.env` currently sets the following keys (values redacted). The same keys must exist on Render production:

```
ADMIN_PHONES
DATABASE_URL
JWT_SECRET
NEXT_PUBLIC_PAYSTACK_PUBLIC_KEY
PAYSTACK_SECRET_KEY
SMTP_FROM
SMTP_HOST
SMTP_PASS
SMTP_PORT
SMTP_USER
TWILIO_ACCOUNT_SID
TWILIO_AUTH_TOKEN
TWILIO_PHONE_NUMBER
```

`render.yaml` additionally declares: `PAYSTACK_WEBHOOK_SECRET`, `TWILIO_WHATSAPP_NUMBER`, `NEXT_PUBLIC_APP_URL`, `CRON_SECRET` (auto-generated).

11. Known issues and bugs

- **Service worker can serve stale chunks in dev.** Mitigated in commit `7b569d7`: dev never registers the SW and actively unregisters any leftover one. Production registration is unchanged.
- **Dev permission bypass is active.** In `src/lib/auth.ts` (`hasPermission` + `hasAnyPermission`) and `src/components/admin/AdminSidebar.tsx` (`canAccess`), `NODE_ENV !== "production"` grants every admin every permission. Grep for `DEV BYPASS` to remove before any QA against staging or prod. Tracked in `memory/project_dev_bypass.md`.
- **Em-dash linter sweep landed in stages.** Commits `5e947b7` → `4380b27` (revert) → `ae23427` (UI copy only). Confirm via `git log` that the head reflects the intended state before relying on copy.
- **`ASK_PER_BOOKING` health visibility short-circuits to no-share.** See TODO note in `src/app/api/partner/bookings/[id]/care-brief/route.ts`. The customer-facing per-booking consent UI has not landed.
- **Frozen care-brief snapshot omits the contraindication rules.** Once a pro acknowledges, `HealthAcknowledgment.snapshot` stores only conditions. Re-displaying the brief after ack shows conditions but not the rule guidance. Live (pre-ack) view is full.
- **Best-effort notifications have no retry queue.** WhatsApp / SMS / email failures only log to console. A durable job queue is not implemented.
- **Per-line-item earnings.** Multi-service commission still computes on the sum, not per item.
- **Auto-deploy applies `prisma db push --accept-data-loss`.** Safe so long as schema changes are additive. A destructive migration would silently drop columns. Use `prisma migrate` when schema changes leave additive territory.
- **OTP delivery in dev logs to console only.** No SMS is dispatched when `NODE_ENV === "development"` — intentional but easy to forget; the dev OTP is in the server log.
- **HMO partner integration is manual.** `attach-policy` on a benefits enrolment is a placeholder until a real provider is integrated.
- **Wallet recompute requires manual trigger today.** If a movement was deleted out-of-band, admins must call `POST /api/admin/wallet/[proId]/recompute`. There is no automatic reconciliation cron.

12. Setup and installation

Prerequisites

- Node.js 20+
- PostgreSQL 14+ (local or hosted)
- Paystack test keys, Twilio sandbox credentials, SMTP credentials (Mailtrap or similar) for OTP / payment / email flows in dev.

Steps

```
# 1. Clone
git clone https://github.com/dodulana-tech/groomee.git
cd groomee

# 2. Install
npm install

# 3. Environment
cp .env.example .env   # if .env.example exists; otherwise create .env manually
# Required keys (see Section 10):
#   DATABASE_URL
#   JWT_SECRET                     # any long random string
#   PAYSTACK_SECRET_KEY            # test key for dev
#   NEXT_PUBLIC_PAYSTACK_PUBLIC_KEY # matching public key
#   PAYSTACK_WEBHOOK_SECRET        # configured in Paystack dashboard
#   TWILIO_ACCOUNT_SID
#   TWILIO_AUTH_TOKEN
#   TWILIO_PHONE_NUMBER
#   TWILIO_WHATSAPP_NUMBER
#   SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASS / SMTP_FROM
#   ADMIN_PHONES                   # comma-separated list of phones allowed to
#   ADMIN_EMAIL                    # super admin email seeded into the DB
#   NEXT_PUBLIC_APP_URL            # e.g. http://localhost:3000
#   CRON_SECRET                    # Bearer token expected by /api/cron/*

# 4. Database
npm run db:push        # create / update schema
npm run db:seed        # seed roles, cities, zones, services, pros
npm run sync:roles     # idempotent role-permission sync (safe to re-run)

# 5. Run
npm run dev            # http://localhost:3000

# 6. Optional sanity
npx tsc --noEmit       # type check
npm run smoke:admin    # smoke admin pages + APIs (requires dev server running)
npm run test:deep      # full end-to-end harness
```

Local admin login

In `NODE_ENV=development` the OTP is logged to the dev server console instead of being delivered. To log in as the seeded super admin:

1. Visit `http://localhost:3000/admin/login`.
2. Enter a phone listed in `ADMIN_PHONES` (or the seeded super-admin phone).
3. Read the OTP from the dev server log (`[DEV] OTP for +234...: 123456`).
4. Submit. Land on `/admin`.

Production deploy

Push to `main` on GitHub. Render auto-deploys with this build command:

```
npm install
  && npx prisma generate
  && npx prisma db push --accept-data-loss
  && npm run sync:roles
  && npm run build
  && cp -r public .next/standalone/
  && cp -r .next/static .next/standalone/.next/
```

Start command: `node .next/standalone/server.js`. Health check path: `/api/health`.

13. Handover notes

Read these first

- `prisma/schema.prisma` — the data model is the most important artifact (57 models, 25 enums).
- `src/lib/permissions.ts` — permission registry + default roles. Adding a new gated feature means adding a key here and granting it to the relevant default role.
- `src/lib/scheduling.ts` — scheduler core: working windows, conflict detection, slot generation, travel-time matrix lookup.
- `src/lib/dispatch.ts` — how a booking finds a pro (auto-dispatch); fires the mobilization release when a pro accepts.
- `src/lib/health.ts` + `src/lib/health-notify.ts` — PHI access patterns; every read writes a `HealthAccessLog` row.
- `src/lib/cities.ts` + `src/lib/city-notify.ts` — city lifecycle and waitlist blast.
- `src/lib/wallet.ts` + `src/lib/wallet-releases.ts` — ledger primitives and the four release flavours; the booking lifecycle hooks into these.
- `src/lib/savings.ts` — Earnings Reserve: skim, withdrawal lifecycle, match-bonus.
- `src/lib/benefits.ts` — plans, eligibility, debit runner (used by the 02:00 UTC cron).
- `src/lib/inventory.ts` — stock primitives + recipe-driven consumption.
- `render.yaml` — deploy contract for the web service + three crons + database.

Things you must do before next prod release

1. Remove the dev permission bypass: grep `DEV BYPASS` and delete both early-return blocks. Verify the sidebar still hides items for non-super-admin roles.
2. Verify `npm run sync:roles` ran on the latest deploy (Render build log will show "Syncing admin role permissions...").
3. Bump `CACHE_NAME` in `public/sw.js` if a UI change must invalidate installed PWAs immediately.
4. Audit any new model for cascade-delete behaviour. `Booking` is the spine; orphan rows here cause UI crashes. `ProWallet` and `ProSavingsAccount` are also load-bearing.
5. Confirm the third cron (`groomee-benefits`) is healthy in Render — missed runs accumulate debits silently.

Workflow conventions

- **Slices, not layers.** Every shipped feature includes schema + lib + API + UI + notifications + error states. Half-built layers are explicitly out of scope; see `memory/feedback_complete_flows.md` .

- **No "groomer".** Use "pro" or "beauty professional" in copy; legacy DB columns and route names still carry "groomer" for back-compat but the user-facing string has been removed.
- **City-driven UI.** Do not hardcode "Lagos" or "Abuja" in new components. Pass `cities[]` from the server and branch on `status`.
- **Multi-service.** Primary service stays on `Booking.serviceId`; additional services live on `BookingItem`. Total duration is cached on `Booking.durationMins` and used by the calendar / availability endpoint.
- **Permission gating.** Sidebar + page + API all enforce permissions. Don't rely on UI gating alone.
- **Wallet first.** New monetary movements should land in the wallet ledger with the right `WalletMovementType`, not directly on `Earning` (legacy) or as ad-hoc fields.
- **Idempotency keys.** Benefits debits and wallet releases must be safe to re-run. Always key on `(entity, period)` or `(bookingId, stage)`.

Where the persistent context lives

Project-level conventions are stored as memories at `/Users/debo/.claude/projects/-Users-debo-groomee/memory/`. Read `MEMORY.md` first for the index. Relevant memories include:

- `project_groomee_overview.md` — brand and market context.
- `feedback_terminology.md` — the "groomer" rejection.
- `feedback_design.md` — glassmorphism guidance.
- `project_vendor_portal.md` — partner portal value proposition.
- `reference_hosting.md` — Render hosting note.
- `project_apprenticeships.md` — the apprenticeship model's strategic role.
- `feedback_complete_flows.md` — slice partitioning rule.
- `project_dev_bypass.md` — the temporary perm bypass to revert.

If you inherit this and want to ship the next slice

1. Plan MECE: enumerate states, surfaces, and notifications first.
2. Sketch the schema delta and seed entries before any UI work.
3. Write the API and the lib helper.
4. Wire the customer / partner / admin surfaces that consume it.
5. Hook notifications and audit trails.
6. Smoke-test on the dev server before commit.
7. Use one commit per slice with a body that explains schema, lib, API, UI, and any drive-by fixes.

End of document — Groomee project handover, 2026-05-21.